

PWM-Based Digital-to-Analog Conversion and Waveform Synthesis on STM32F4 MCU

Roston Smith
SMTROS022

Abstract—This electronic document presents the implementation of a pulse-width modulation (PWM) based digital-to-analog converter system using an STM32F4 microcontroller for real-time waveform synthesis and audio signal generation. The system utilizes Direct Memory Access (DMA) to transfer values from lookup tables (LUTs) to the PWM register on set intervals to generate the output. This output is then fed through a simple, single transistor amplifier circuit and output through a small speaker.

Keywords— *PWM, DAC, STM32, DMA, embedded systems, waveform synthesis, energy-adaptive resampling, LUT*

I. INTRODUCTION

DACs serve as fundamental interfaces between digital processing systems and analog environments. While dedicated DACs are often built in to embedded systems and offer higher precision, microcontroller-based PWM DACs offer a more cost-effective solution where “moderate” resolution is viable.

PWM-based DACs operate on the principle that you can write pre-computed sample values of a processed signal from LUTs on precise intervals to a GPIO pin and functionally generate the desired signal.

This work addresses the challenges of implementing high-quality audio synthesis on resource-constrained microcontrollers. The STM32F446 used in this lab has 128KB of SRAM and 256KB of Flash storage. Given that the sample waveforms to be replicated in the lab were sampled at 44.1kHz and are over 10 seconds long, it is impossible to store let alone process all the points in the files. As such energy-adaptive resampling was used to select the best 1 second segment of the file and then lessen the number of points in that segment while maintaining as much of the signal as possible.

The primary objectives of the lab are to: generate LUTs for the waveforms to be generated, implement DMA-based transfers and PWM-based output for real-time operation, designing and building an amplifier circuit and finally outputting the signals through a simple speaker.

II. METHODOLOGY

A. Hardware Platform

The implementation utilized an STM32F446RET6 microcontroller featuring:

- ARM Cortex-M4F core with DSP instructions and FPU
- 180 MHz maximum clock (operated at 16 MHz HSI)
- 128 KB SRAM, 512 KB Flash memory
- Advanced-control timers (TIM2: 32-bit, TIM3: 16-bit)
- 12-channel DMA controller with request multiplexer

Clock Configuration:

The internal high-speed oscillator (HSI) provided a 16 MHz system clock. While the STM32F446 supports up to 180 MHz with PLL, the HSI configuration simplified development and provided adequate performance for this application.

Timer Configuration:

TIM3 (PWM Generation):

```
chtim3.Init.Prescaler = 0;
htim3.Init.Period = 4095;
htim3.Init.CounterMode =
TIM_COUNTERMODE_UP;
```

This configuration yields:

- PWM frequency: $16 \text{ MHz} / 4096 = 3.906 \text{ kHz}$
- Resolution: 12 bits (4096 levels)
- Duty cycle range: 0% to 100%


```

        DestAddress,
// Destination
        NS);
// Transfer count
    __HAL_TIM_ENABLE_DMA(&htim2,
TIM_DMA_CC1); // Enable DMA requests
}
3) Interrupt Service Layer:
Handles user input and waveform switching.
void EXTI0_IRQHandler(void) {
    uint32_t current_tick =
HAL_GetTick();

    // Debouncing
    if (current_tick - last_button_press
< 1000) {

HAL_GPIO_EXTI_IRQHandler(Button0_Pin);
    return;
    last_button_press = current_tick;

    // Stop current DMA transfer
    __HAL_TIM_DISABLE_DMA(&htim2,
TIM_DMA_CC1);
    HAL_DMA_Abort_IT(&hdma_tim2_ch1);

    // Cycle to next waveform
    current_waveform = (current_waveform
+ 1) % 6;

    // Select new LUT and parameters
    switch(current_waveform) {
        case 0: /* Sine */ break;
        case 1: /* Sawtooth */ break;
        // ... additional cases
    }

    // Recalculate timer period
    TIM2_Ticks = TIM2CLK / (ns_to_use *
frequency);
    htim2.Init.Period = TIM2_Ticks - 1;
    HAL_TIM_Base_Init(&htim2);

    // Update display
    lcd_command(CLEAR);
    lcd_putstring(waveform_name);

    // Restart DMA with new LUT
    HAL_DMA_Start_IT(&hdma_tim2_ch1,
(uint32_t)selected_lut,
        DestAddress,
        ns_to_use);
    __HAL_TIM_ENABLE_DMA(&htim2,
TIM_DMA_CC1);

```

```

HAL_GPIO_EXTI_IRQHandler(Button0_Pin);
}

```

Debouncing Implementation:

Mechanical switch contacts exhibit bounce, creating multiple transitions during a single press event. A time-based debounce filter with 1000 ms threshold prevents spurious triggers:

```

if (current_tick - last_button_press <
1000)
return;

```

While 1000 ms may seem excessive for typical bounce duration (5-20 ms), it provides robust rejection and acceptable user experience for this application.

D. Lookup Table Generation

As this course and subsequently this lab are not concerned with signal processing or python coding, AI, in the form of Claude, was utilized to process the .wav files and generate the necessary LUTs for all 6 signals. Through various prompts, a complete .py file was generated that sampled the files, found the best 1 second segment, and sampled 8192 values to be added to the LUT for each .wav file.

III. RESULTS AND DISCUSSION

A. Synthetic Waveform Characterisation

The 3 generic waveforms performed exceptionally well with a F_SIGNAL value of 1Hz. Each signal could be distinctly recognised and the values parsed smoothly from the LUTs. As for the 3 wav files, the 1 second audio segment with around a fifth of the original resolution resulted in a signal and thus output sound that was hard to recognise as the original segment. However, given the severe lack of resources, in this case memory restrictions, the “DIY” DAC performed well and the signal can be distinctly seen as it should appear using an Oscilloscope.

B. Amplifier Performance

At the time of this report, the Amplifier was yet to be tested with a signal generator, however, that being said the sound output from the amplifier when given the MCU output appears to be adequate for the purposes of this lab.

IV. CONCLUSION

This work successfully demonstrated a complete PWM-based DAC system on an STM32F446 microcontroller, achieving real-time waveform synthesis and compressed audio playback within severe memory constraints. The implementation synthesized three mathematically accurate waveforms at 1024-sample resolution and reproduced three audio signals at 8192-sample resolution using 108 KB of 128 KB available SRAM.

Key findings include:

Frequency Accuracy: Synthetic waveforms exhibited a slight frequency error due to timer quantization, acceptable for audio applications but improvable via higher clock frequencies or software compensation.

DMA Performance: Direct Memory Access enabled low CPU utilization during waveform generation, validating the use of DMA for deterministic real-time signal synthesis.

Amplifier Characterization: The single-transistor common-emitter amplifier seemed to prove adequate for educational and low-fidelity applications such as this lab.

Memory-Quality Trade-off: The shortage of memory resulted in a significant quality trade-off and the output signal could not be near as defined as the original. As such, almost all of the original files were lost.

Real-Time Performance: Low waveform switching latency demonstrated system responsiveness sufficient for interactive applications.

Future enhancements could include:

PLL Clock Configuration: Operating at 180 MHz would, given no restriction on memory, allow for the entire original file to be smoothly output to the PWM DAC.

Improved Amplifier: A more sophisticated amplifier would reduce distortion, improve efficiency, and increase output power.

Adaptive Bit Depth: Allocating more bits to high-energy samples and fewer to low-energy samples could reduce memory by 20-30% with minimal perceptual impact.

True DAC Integration: External 12-bit DAC would eliminate PWM artifacts entirely while maintaining DMA-based operation.

This implementation confirms that PWM-based DACs, coupled with intelligent compression algorithms and efficient memory management, provide viable solutions for embedded audio systems where cost, complexity, and power consumption must be minimized. The techniques demonstrated here apply broadly to resource-constrained applications including IoT devices, wearables, battery-powered instruments, and educational platforms.

The project achieved its objectives of demonstrating practical embedded audio synthesis while providing quantitative analysis of the trade-offs inherent in real-world engineering. The balance between theoretical understanding and practical implementation reflects the realities of embedded systems development, where elegant solutions must accommodate hardware limitations, component availability, and economic constraints.

AI USAGE STATEMENT

AI was utilized in generating initial ideas for the code implementation as well as the entire .py signal processing implementation. Once the code was finalized, AI was used to format and include necessary comments. AI was also used to aid with inspiration for the reporting and formatting and spell checking the report.

REFERENCES

- [1] STMicroelectronics, "PM0214: STM32 Cortex-M4 Programming Manual," 2023. [Online]. Available: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf
- [2] J. Riyaz, "Simple Audio Amplifier Using Single Transistor," Instructables, Mar. (?) [Online]. Available: <https://www.instructables.com/Simple-Audio-Amplifier-Using-Single-Transistor/> [Accessed: Oct. 1, 2025]

APPENDIX A: FULL CODE SOLUTION

```
c/*
 * GitHub Repository:
 * https://github.com/RostonSmith/EEE3096S-2025-
 * Group_98.git
 * Group Number: 98
 * Members: SMTROS022
 */

/* USER CODE BEGIN Header */
/**

*****
*****
 * @file      : main.c
 * @brief     : PWM-based DAC implementation with
DMA-driven waveform
 *             synthesis for STM32F446

*****
*****

 * @attention

 *
 * Copyright (c) 2025 STMicroelectronics.
 * All rights reserved.
 *

*****
*****

 */
/* USER CODE END Header */

/* Includes -----
-----*/
#include "main.h"
#include <stdio.h>
#include "stm32f4xx.h"
#include "lcd_stm32f4.h"

/* Private defines -----
-----*/
/* USER CODE BEGIN PD */
#define NS      1024    // Number of samples for
waveforms
#define NS_WAVE  8192    // Number of samples for
audio
#define TIM2CLK  16000000 // STM32 clock frequency
(16 MHz)
#define F_SIGNAL 1        // Base frequency (Hz)
```

```

/* USER CODE END PD */

/* Private variables -----*/
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
DMA_HandleTypeDef hdma_tim2_ch1;

/* USER CODE BEGIN PV */

// State variables
volatile uint8_t current_waveform = 0;
volatile uint32_t last_button_press = 0;
uint32_t current_tick;
uint32_t *selected_lut;
char *waveform_name;
float frequency;
uint32_t ns_to_use;

// Waveform LUTs (1024 samples each = 12 KB total)
uint32_t Sin_LUT[NS] = {
    // [Insert 1024 sine wave values here]
    2048, 2060, 2073, 2085, // ... (1024 values total)
};

uint32_t Saw_LUT[NS] = {
    // [Insert 1024 sawtooth values here]
    0, 4, 8, 12, // ... (1024 values total)
};

uint32_t Triangle_LUT[NS] = {
    // [Insert 1024 triangle values here]
    0, 8, 16, 24, // ... (1024 values total)
};

// Audio LUTs (8192 samples each = 96 KB total)
uint32_t Piano_LUT[NS_WAVE] = {
    // [Insert 8192 piano values here]
};

uint32_t Guitar_LUT[NS_WAVE] = {
    // [Insert 8192 guitar values here]
};

uint32_t Drum_LUT[NS_WAVE] = {
    // [Insert 8192 drum values here]
};

// Timer configuration
uint32_t TIM2_Ticks = TIM2CLK / (NS * F_SIGNAL);
uint32_t DestAddress = (uint32_t) &(TIM3->CCR3);

/* USER CODE END PV */

/* Function prototypes -----*/
void SystemClock_Config(void);

```

```

static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM3_Init(void);
void EXTI0_IRQHandler(void);

/* Main function -----*/
int main(void)
{
    /* Initialize HAL and peripherals */
    HAL_Init();
    SystemClock_Config();

    /* Initialize LCD */
    HAL_Delay(100);
    init_LCD();
    lcd_command(CLEAR);
    lcd_putstr("Sine");
    HAL_Delay(100);

    /* Initialize GPIO, DMA, Timers */
    MX_GPIO_Init();
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4,
        GPIO_PIN_RESET);
    MX_DMA_Init();
    MX_TIM2_Init();
    MX_TIM3_Init();

    /* Start PWM generation */
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);

    /* Start DMA trigger timer */
    HAL_TIM_OC_Start(&htim2, TIM_CHANNEL_1);

    /* Configure and start DMA transfer */
    HAL_DMA_Start_IT(&hdma_tim2_ch1,
        (uint32_t)Sin_LUT,
        DestAddress, NS);

    /* Update LCD */
    lcd_command(CLEAR);
    lcd_putstr("Sine");

    /* Enable DMA requests from TIM2 */
    __HAL_TIM_ENABLE_DMA(&htim2,
        TIM_DMA_CC1);

    /* Main loop */
    while (1)
    {
        /* Empty - all processing handled by DMA and interrupts */
    }
}

/* Interrupt handler for button press */

```

```

void EXTI0_IRQHandler(void)
{
    current_tick = HAL_GetTick();

    /* Debouncing: ignore presses within 1000ms */
    if (current_tick - last_button_press < 1000) {
        HAL_GPIO_EXTI_IRQHandler(Button0_Pin);
        return;
    }

    last_button_press = current_tick;

    /* Stop current DMA transfer */
    __HAL_TIM_DISABLE_DMA(&htim2,
TIM_DMA_CC1);
    HAL_DMA_Abort_IT(&hdma_tim2_ch1);

    /* Cycle to next waveform */
    current_waveform = (current_waveform + 1) % 6;

    /* Select appropriate LUT and parameters */
    switch(current_waveform) {
        case 0: // Sine
            selected_lut = Sin_LUT;
            waveform_name = "Sine";
            frequency = F_SIGNAL;
            ns_to_use = NS;
            break;

        case 1: // Sawtooth
            selected_lut = Saw_LUT;
            waveform_name = "Sawtooth";
            frequency = F_SIGNAL;
            ns_to_use = NS;
            break;

        case 2: // Triangle
            selected_lut = Triangle_LUT;
            waveform_name = "Triangle";
            frequency = F_SIGNAL;
            ns_to_use = NS;
            break;

        case 3: // Piano
            selected_lut = Piano_LUT;
            waveform_name = "Piano";
            frequency = F_SIGNAL;

            ns_to_use = NS_WAVE;
            break;

        case 4: // Guitar
            selected_lut = Guitar_LUT;
            waveform_name = "Guitar";
            frequency = F_SIGNAL;
            ns_to_use = NS_WAVE;
            break;

        case 5: // Drum
            selected_lut = Drum_LUT;
            waveform_name = "Drums";
            frequency = F_SIGNAL;
            ns_to_use = NS_WAVE;
            break;

        default:
            selected_lut = Sin_LUT;
            waveform_name = "Sine";
            frequency = F_SIGNAL;
            ns_to_use = NS;
            break;
    }

    /* Recalculate timer period for new waveform */
    TIM2_Ticks = TIM2CLK / (ns_to_use * frequency);
    htim2.Init.Period = TIM2_Ticks - 1;
    HAL_TIM_Base_Init(&htim2);

    /* Update LCD display */
    lcd_command(CLEAR);
    lcd_putstr(waveform_name);

    /* Restart DMA with new LUT */
    HAL_DMA_Start_IT(&hdma_tim2_ch1,
(uint32_t)selected_lut,
        DestAddress, ns_to_use);

    /* Re-enable DMA requests */
    __HAL_TIM_ENABLE_DMA(&htim2,
TIM_DMA_CC1);

    /* Clear interrupt flags */
    HAL_GPIO_EXTI_IRQHandler(Button0_Pin);
}

```